AI Algorithms in Robotic Navigation

https://gemini.google.com/app/0cc66723e86936d2

User prompt: What is that AI algorithm in robotics? the one where based on the visual recognition of the balls, it control the robot to the positions in the playing field such that you have the most optimal ball collection strategy. It also includes avoidance detection to prevent you from colliding with other robots. And the path continuously updates itself

Response: In modern robotics, there isn't just one monolithic "AI algorithm" that handles all of those tasks at once. Instead, what you are describing is a classic **autonomous navigation pipeline**. In competitions like RoboCup or FIRST Robotics, engineers achieve this behavior by stacking three distinct algorithmic layers working in real-time:

1. Visual Recognition (The Eyes)

To recognize the balls on the field, the robot uses a computer vision algorithm, most commonly **YOLO (You Only Look Once)** or a custom **Convolutional Neural Network (CNN)**. These algorithms process the camera feed frame-by-frame to identify the coordinates of every ball on the field instantly.

2. Optimal Ball Collection (The Master Plan)

Once the robot knows where the balls are, it has to decide which order to collect them in. This is fundamentally a routing algorithm solving the **Traveling Salesperson Problem (TSP)** or utilizing a **Nearest Neighbor** approach. The algorithm calculates the most efficient sequence to collect multiple targets, minimizing time and distance based on the robot's current position and heading.

3. Continuous Path Update & Avoidance (The Reflexes)

This is the specific part of the pipeline you are likely thinking of. Once the robot knows *where* to go, a **Local Path Planner** takes over. If the path continuously updates itself to dodge moving obstacles (like other robots) while pushing toward the goal, it is almost certainly using one of these three algorithms:

- **Artificial Potential Fields (APF):** This algorithm treats the robot like a charged particle. The target ball acts as an "attractive field" (pulling the robot toward it), while other robots and walls act as "repulsive fields" (pushing the robot away). As obstacles move, the invisible forces shift, causing the robot's path to continuously curve and update itself.
- **Dynamic Window Approach (DWA):** The robot calculates hundreds of possible short-term trajectories (velocities and steering angles) multiple times a second. It scores them based on progress toward the ball, speed, and clearance from other robots, constantly updating its route.
- *D Lite (Dynamic A*): Unlike standard A* (which calculates a path once), D* Lite operates in reverse and instantly replans only the necessary parts of the route the moment its sensors detect a new obstacle blocking the way.

*(Note: In cutting-edge research, some teams use **Deep Reinforcement Learning**, where a single neural network is trained end-to-end to take in visual data and output steering commands, balancing the "rewards" of collecting balls and avoiding collisions simultaneously.)*

To see how the continuous updating behavior actually works in real-time, here is a simulation of the **Artificial Potential Fields** algorithm:

Dynamic Path Planning and Obstacle Avoidance Methodologies Watching how these path planning methodologies are implemented across various robotic platforms illustrates exactly how they adjust routes continuously in dynamic environments.

Sources

1. arXiv:2309.03530v1 [cs.CV] 7 Sep 2023

User prompt: I am running it with a raspberry pi 4

Response: Running this exact autonomous pipeline on a Raspberry Pi 4 is the standard approach for student and hobbyist robotics. The Pi's quad-core Cortex-A72 CPU is more than capable of handling the routing and path-planning math, but because it lacks a dedicated GPU, you will need to optimize the visual recognition layer so it doesn't throttle your framerate.

Here is how you adapt that three-layer architecture for a Pi 4:

1. Optimizing the Vision Layer (The Bottleneck)

If you try to run a standard, unoptimized deep learning model (like YOLOv8) directly on the Pi 4, it will crawl at roughly 2 to 4 frames per second (FPS). You have three realistic workarounds:

- **The Lightweight Approach (OpenCV):** If the balls you are collecting are a solid, distinct color (like bright green tennis balls or orange ping-pong balls), skip AI entirely. Use classic OpenCV HSV color masking and contour detection. This runs effortlessly on the Pi 4 CPU, giving you real-time tracking.
- **The Quantized AI Approach (TFLite / NCNN):** If you must use neural networks to recognize complex ball patterns, use a "nano" model (YOLOv8n) and convert it to an INT8 quantized format using TensorFlow Lite or NCNN. By compressing the model math to 8-bit integers, you can push the Pi 4 to achieve roughly 10 to 20 FPS.
- **The Hardware Approach (Google Coral):** If you want full-speed AI, plug a Google Coral Edge TPU into one of the Pi's USB 3.0 ports. This acts as an external GPU strictly for neural networks, allowing heavy YOLO models to run at 30+ FPS without stressing the Pi's CPU.

2. The Master Plan (TSP)

The Traveling Salesperson Problem (TSP) or nearest-neighbor logic is computationally cheap for small datasets. If you have 5 to 20 balls on the field, a basic Python or C++ script will calculate the optimal collection sequence in less than a millisecond. The Pi 4 will handle this effortlessly on a single core.

3. Continuous Path Avoidance (ROS 2)

You do not need to write complex avoidance algorithms like Artificial Potential Fields or the Dynamic Window Approach (DWA) from scratch.

The industry standard for a Pi 4 is to install **ROS 2 (Robot Operating System)**. ROS 2 includes a package called **Nav2** (Navigation 2), which has built-in local planners that use algorithms like DWA out of the box. You simply feed Nav2 your camera data and your optimal ball sequence, and it will continuously calculate the motor speeds required to drive to the targets while dynamically curving around moving obstacles.

User prompt: I dont want to use ROS2. The motors and all are controlled using ESP32

Response: If you are skipping ROS 2 and using an ESP32 for the low-level motor control, you are building a custom **distributed architecture**. This is a great approach because it plays to the strengths of both boards: the Raspberry Pi handles the heavy math (vision and pathfinding), while the ESP32 handles the strict real-time timing required for hardware control (PWM signals, encoders, and PID loops).

Here is exactly how you structure that pipeline from scratch.

1. The Communication Bridge (Serial / UART)

The Pi and the ESP32 need to talk continuously. The most robust way to do this is via a USB cable connecting the two, using standard UART Serial communication.

- **The Pi's Job:** It runs a Python script (using the `pyserial` library). Every few milliseconds, it calculates the desired velocity and steering angle based on the vision data and sends a simple string command over serial (e.g., `<V:150, S:-20>`, meaning velocity 150, steering -20).
- **The ESP32's Job:** It runs C++ code (Arduino IDE or ESP-IDF) that constantly listens to the serial port. It parses that incoming string, breaks out the numbers, and translates them into motor commands.

2. Motor Control & PID on the ESP32

You cannot just tell a motor to "spin at speed 150." Battery voltage drops and physical friction change constantly. To ensure the robot actually drives straight and turns accurately, the ESP32 must run a **PID Controller (Proportional, Integral, Derivative)** for each motor wheel.

1. **Read Encoders:** The ESP32 reads the hardware encoders on the motors to know exactly how fast the wheels are *actually* spinning.
2. **Calculate Error:** It compares the *actual* speed to the *target* speed sent by the Raspberry Pi.
3. **Adjust PWM:** The PID algorithm calculates exactly how much to increase or decrease the PWM voltage to force the motor to match the target speed, updating hundreds of times per second.

3. Implementing Avoidance (Artificial Potential Fields in Python)

Since you aren't using ROS, the easiest continuous-update algorithm to write yourself in Python on the Raspberry Pi is **Artificial Potential Fields (APF)**. The math translates very cleanly into a loop:

1. **Attraction Vector:** Calculate a vector pointing from the robot's current (X,Y) to the target ball's (X,Y). Multiply it by an attraction weight.
2. **Repulsion Vectors:** For every obstacle detected by the camera, calculate a vector pointing *away* from it. Multiply this by a repulsion weight that gets exponentially stronger as the obstacle gets closer.
3. **Net Force:** Add the vectors together. The resulting vector gives you your immediate target heading and speed, which you send over serial to the ESP32.

To understand exactly how the ESP32 keeps the motors behaving accurately while receiving those commands from the Pi, here is a simulation of how the PID controller works under the hood:

User prompt: How should I format and parse the serial data strings being sent between the Python script on the Raspberry Pi and the C++ code on the ESP32?

Response: The most robust and industry-standard way to handle this on hobbyist hardware is to use **Start and End Markers** with comma-separated values.

When you boot up the Pi and ESP32, they might start mid-conversation. If the Pi sends `150,-20` but the ESP32 only catches `50,-20`, your robot will violently jerk. Wrapping your data in `<` and `>` ensures the ESP32 only executes a command when it receives a complete, valid packet.

Here is the best-practice implementation for a format like `<M,150,-20>` (where `M` is the motor command, `150` is velocity, and `-20` is steering).

1. The Python Sender (Raspberry Pi)

The Python side is very simple. It takes your calculated values, formats them into the bracketed string, encodes them to bytes, and blasts them over the serial port.

```
import serial
import time

# Update '/dev/ttyUSB0' to match your Pi's actual USB port
esp32 = serial.Serial('/dev/ttyUSB0', baudrate=115200, timeout=1)
time.sleep(2) # Give the ESP32 a moment to reset upon connection

def send_motor_command(velocity, steering):
    # Format: <M,velocity,steering>
    packet = f"<M,{velocity},{steering}>"
    esp32.write(packet.encode('utf-8'))

# Example usage inside your main vision loop:
while True:
    # ... vision and APF logic happens here ...
    v = 150  # Calculated velocity
    s = -20  # Calculated steering

    send_motor_command(v, s)
    time.sleep(0.05) # Send commands at 20Hz
```

2. The C++ Receiver (ESP32)

The ESP32 side must be **non-blocking**. You cannot use `Serial.readString()` or `delay()`, because while the ESP32 is waiting for a string, it isn't updating your motors' PID loops.

Instead, use a character-by-character state machine. It grabs one byte at a time as it arrives in the hardware buffer, and only attempts to parse it when it sees the `>` character.

```
const byte numChars = 32;
char receivedChars[numChars];
boolean newData = false;

// Variables to hold the parsed data
char commandType;
int targetVelocity = 0;
int targetSteering = 0;

void setup() {
    Serial.begin(115200);
}
```

```
void loop() {
    recvWithStartEndMarkers();

    if (newData == true) {
        parseData();
        newData = false;
    }

    // --> YOUR FAST PID MOTOR CONTROL CODE GOES HERE <--
    // It runs freely at maximum speed using targetVelocity
    // and targetSteering without being blocked by Serial!
}

// 1. NON-BLOCKING READ FUNCTION
void recvWithStartEndMarkers() {
    static boolean recvInProgress = false;
    static byte ndx = 0;
    char startMarker = '<';
    char endMarker = '>';
    char rc;

    while (Serial.available() > 0 && newData == false) {
        rc = Serial.read();

        if (recvInProgress == true) {
            if (rc != endMarker) {
                receivedChars[ndx] = rc;
                ndx++;
                if (ndx >= numChars) ndx = numChars - 1; // Prevent overflow
            } else {
                receivedChars[ndx] = '\0'; // Terminate the string
                recvInProgress = false;
                ndx = 0;
                newData = true;
            }
        } else if (rc == startMarker) {
            recvInProgress = true;
        }
    }
}

// 2. PARSING FUNCTION
void parseData() {
    // strtok() splits the string at the commas
    char * strtokIndx;

    strtokIndx = strtok(receivedChars, ","); // Get the first part (the 'M')
    commandType = strtokIndx[0];

    strtokIndx = strtok(NULL, ","); // Continue to the first number
    targetVelocity = atoi(strtokIndx); // atoi converts string to integer

    strtokIndx = strtok(NULL, ","); // Continue to the second number
    targetSteering = atoi(strtokIndx);

    // Optional: Print back to the Pi for debugging
    // Serial.print("Ack:"); Serial.print(targetVelocity); Serial.print(","); Serial.println(targetSteering);
}
```

Why this architecture wins

By using `strtok` (string tokenization) and this non-blocking read loop, you get two massive benefits:

1. **Zero lag:** The ESP32's `loop()` will run thousands of times per second, ensuring your PID loops are incredibly responsive, only pausing for a fraction of a microsecond to grab a serial byte when one arrives.
2. **Scalability:** If you later decide to add a servo mechanism for a ball flipper, you just send `<F,90>` from the Pi, and update `parseData()` to check if `commandType == 'F'` instead of `'M'`.
